C Programming Basics 💻

C is a general-purpose, procedural programming language widely used for system software, embedded systems, and applications. It is known for its efficiency, low-level memory access, and portability. A C program must be compiled before it can be executed.

Basic Program Structure

A typical C program has a specific structure:

- **Header Files**: These contain declarations for functions and macros. They are included at the beginning of the program using the `#include` directive (e.g., `#include <stdio.h>`). The `stdio.h` header file provides standard input/output functions like `printf()` and `scanf()`.
- **`main()` function**: This is the entry point of every C program. Execution always begins here. It has a return type (usually `int`) and takes arguments. The return value indicates the program's exit status.
- **Statements**: These are instructions that perform specific tasks. They are enclosed within the `main()` function or other functions and must end with a semicolon `;`.
- **Comments**: Used to explain code. They are ignored by the compiler.
  - Single-line comments start with `//`.
  - Multi-line comments are enclosed in `/* ... */`.

Pointers 📍

A **pointer** is a variable that stores the memory address of another variable. Instead of holding a data value (like an integer or a character), it holds the location where that data is stored. Pointers are fundamental to C programming and are used for dynamic memory allocation, efficient array manipulation, and passing arguments to functions by reference.

Key Concepts

- **Declaration**: A pointer variable is declared with an asterisk `*`. For example, `int *p;` declares a pointer `p` that can hold the address of an integer variable.
- **Initialization**: A pointer is initialized with the memory address of another variable using the **address-of operator** `&`. For example, if `int x = 10;`, then `p = &x;` assigns the memory address of `x` to the pointer `p`.
- **Dereferencing**: To access the value at the address stored in a pointer, you use the **dereference operator** `*`. For example, `*p` will give you the value stored at the address `p` points to (which is `10` in this case).

Example: Pointers in C

C
```c
#include <stdio.h>

int main() {
    int value = 100; // A regular integer variable
    int *ptr;        // A pointer variable to an integer
```

```
    // 1. Assign the address of 'value' to the pointer 'ptr'
    ptr = &value;

    printf("Value of the variable 'value': %d\n", value);
    printf("Address of the variable 'value': %p\n", &value); // %p is
used for printing addresses
    printf("Value stored in the pointer 'ptr': %p\n", ptr);

    // 2. Dereferencing the pointer to get the value it points to
    printf("Value accessed through the pointer '*ptr': %d\n", *ptr);

    // 3. Changing the value using the pointer
    *ptr = 200;
    printf("New value of 'value' after changing it via the pointer:
%d\n", value);

    return 0;
}
```